

АКАДЕМИЈА ТЕХНИЧКО-УМЕТНИЧКИХ СТРУКОВНИХ СТУДИЈА
БЕОГРАД

ОДСЕК ВИСОКА ШКОЛА ЕЛЕКТРОТЕХНИКЕ И РАЧУНАРСТВА

Пантелић Никола

**Портал за доставу хране са применом алгоритма A^* за
оптимизовање руте**

- завршни рад -



Београд, септембар 2026.

Кандидат: Пантелић Никола

Број индекса: ИС-10/23

Студијски програм: Информациони системи

Тема: Портал за доставу хране са применом алгоритма A^* за оптимизовање руте

Основни задаци:

- 1. Основни концепти примењених технологија**
- 2. Дефинисање програмских целина и повезивање са подацима**
- 3. Програмско решење и тестирање**

Ментор:

Београд, септембар 2026. године.

др Зоран Ћировић, проф. ВИШЕР

САЖЕТАК

У раду је представљена архитектура и серверска реализација портала Voro за доставу хране са локалном применом алгоритма A*. Систем повезује купца, ресторан, достављача и администратора преко заједничког програмског интерфејса. Сервер је организован као модуларни монолит са слојевима за обраду захтева, пословну логику и приступ подацима. PostgreSQL обезбеђује трајно стање и трансакције, Redis подржава присутност и доделу, а Socket.IO омогућава праћење промена. Избор достављача заснива се на доступности, Хаверсиновом растојању и оптерећењу. A* се користи за други задатак: проналажење пута минималног процењеног времена у усмереном графу изграђеном из OpenStreetMap података. Објашњени су тежине, хеуристика, приоритетни ред, реконструкција линије и процена трајања обе деонице испоруке. Посебно су размотрени статусна процена поруџбине, квалитет локације и локална симулација кретања. Провера језгра потврдила је избор јефтиније путање и исход без доступног пута. Интеграциона провера над подацима Београда вратила је A* линију дужине 995 m са 81 тачком. Резултати показују функционалну повезаност алгоритма и backend-a, уз ограничења величине графа, приступних сегмената и усвојеног модела брзине.

Кључне речи: достава хране, архитектура, A*, додела достављача, процена времена.

ABSTRACT

This thesis presents the architecture and backend implementation of Voro, a food delivery portal with local A* routing. Customers, restaurants, couriers and administrators use a shared application interface. The backend is a modular monolith with separate request handling, business logic and data access layers. PostgreSQL maintains persistent state and transactions, Redis supports presence and dispatch, and Socket.IO distributes tracking updates. Courier selection combines availability, Haversine distance and workload. A* solves a separate problem: finding a path with minimum estimated travel time in a directed graph built from OpenStreetMap data. The thesis explains edge weights, the heuristic, the priority queue, path reconstruction and travel estimates for both delivery legs. It also examines order status estimates, location quality and local movement simulation. The algorithm test confirmed the lower cost path and the no-path case. An integration check using road data from Belgrade returned a 995 m A* route containing 81 coordinates. These results demonstrate the integration of routing with backend services while identifying limitations related to graph coverage, endpoint connections and the assumed speed model.

Keywords: food delivery, architecture, A*, courier dispatch, travel time estimation.

САДРЖАЈ

1. Увод.....	1
2. Архитектура система	2
2.1. Учесници и одговорности	2
2.2. Модуларна и слојевита организација.....	3
2.3. Процеси и комуникација	4
3. Организација backend-а и обрада захтева.....	5
3.1. Улазни HTTP слој	5
3.2. Аутентификација и овлашћења	5
3.3. Креирање поруџбине	5
3.4. Животни циклуси и серверски интерфејси	6
4. Подаци и инфраструктура.....	8
4.1. Релациони модел.....	8
4.2. Трансакције и конкурентни захтеви	8
4.3. Улога система Redis.....	9
5. Алгоритам проналажења достављача	10
5.1. Покретање и избор кандидата	10
5.2. Растојање и оцена кандидата	10
5.3. Понуде и поновни покушаји.....	11
6. A* алгоритам и формирање руте.....	12
6.1. Модел путне мреже.....	12
6.2. Избор подручја и учитавање графа	12
6.3. Проходност, смер и брзина	13
6.4. Трошак пута и хеуристика	14
6.5. Структуре података и кораци претраге	14
6.6. Повезивање координата и резултат.....	15
7. Процена времена и дужине испоруке	17
7.1. Време вожње по A* рути.....	17
7.2. Процена понуде и активне испоруке	17
7.3. Процена поруџбине пре живог праћења.....	18
8. Локација и праћење испоруке.....	19
8.1. Пријем и провера локације	19
8.2. Објављивање догађаја	19

8.3. Локална симулација кретања	20
9. Покретање и провера решења	21
9.1. Организација окружења	21
9.2. Провера језгра алгоритма	21
9.3. Провера над стварним географским подацима	21
9.4. Ограничења и правци унапређења	22
10. Закључак	23
Литература	24

СПИСАК СЛИКА

Слика 2.1. Функционалне могућности достављача.	2
Слика 2.2. Архитектура серверског система и његове зависности.	3
Слика 3.1. Креирање поруџбине и покретање доделе након прихватања.	6
Слика 5.1. Ток филтрирања, рангирања и понуде испоруке.	11
Слика 6.1. Секвенца локалног A* рутирања над OSM подацима.	16
Слика 8.1. Обрада локације и објављивање стања испоруке.	20

СПИСАК ТАБЕЛА

Табела 3.1. Основна стања пословних и оперативних записа.	7
Табела 3.2. Кључни серверски интерфејси за извршење испоруке.	7
Табела 4.1. Улоге главних група табела.	8
Табела 5.1. Илустративни прорачун оцене достављача.	11
Табела 6.1. Изабране подразумеване брзине у моделу рутирања.	13
Табела 7.1. Основни статусни опсези пре додатака за припрему.	18
Табела 9.1. Резултат једне интеграционе провере A* руте.	21

СПИСАК СКРАЋЕНИЦА

API — програмски интерфејс апликације.

HTTP — протокол за размену захтева и одговора.

JSON — текстуални формат за размену података.

JWT — токен за пренос потписаних тврдњи о идентитету.

SQL — језик за рад са релационим подацима.

OSM — OpenStreetMap, извор отворених географских података.

ETA — процењено време доласка.

GPS — систем за сателитско позиционирање.

TTL — рок важења привременог записа.

CORS — правила приступа из различитих веб извора.

1. УВОД

Портал Voro повезује купца, ресторан и достављача током наручивања и испоруке хране. Купац формира поруџбину, ресторан потврђује њен пријем и припрема храну, а достављач прихвата понуду и извршава превоз. Сервер координира ове активности, чува пословне податке и проверава услове за промену стања. За успешну испоруку потребно је истовремено решити организацију поруџбине, избор расположивог достављача, проналажење проходне путне везе и праћење кретања. Због тога су архитектура и серверска логика основа целокупног решења.

Предмет рада је реализација серверског дела портала са алгоритмом A* за оптимизовање руте. Имплементација преузима географске податке из пројекта OpenStreetMap, од њих формира усмерени тежински граф и у самом серверу тражи пут минималног процењеног времена. Резултат садржи координате путне линије, укупну дужину и процену трајања вожње. Исти поступак користи се при прегледу понуде достављачу, навигацији активне доставе и локалној симулацији кретања. Тако је алгоритам део извршног тока апликације и има непосредан утицај на податке које добијају учесници испоруке.

Избор достављача и избор пута представљају два одвојена проблема. Први одговара на питање коме треба понудити поруџбину, а други којим улицама изабрани достављач може стићи до одредишта. Voro за први проблем користи географску удаљеност, доступност и оптерећење достављача. За други користи A*. Ово раздвајање омогућава да се велики број потенцијалних кандидата прво обради једноставним рачунањем растојања, док се детаљан путни граф учитава када је потребна конкретна рута.

Циљ рада је да објасни везу између архитектуре, података и алгоритама. Посебно се разматрају пријем HTTP захтева, слојеви пословне логике и приступа бази, трансакцијско прихватање понуде, конструисање графа и израчунавање времена. Поред исправног тока, обрађују се случајеви недоступног достављача, истекле понуде, застареле локације и недоступног географског извора. Ови случајеви су важни јер се испорука одвија током времена и зависи од више независних учесника.

Опис обухвата архитектуру и серверску реализацију, релациони модел, позадинске процесе и проверу алгоритма. Клијентске апликације посматрају се као корисници серверских интерфејса. Њихов визуелни изглед није неопходан за објашњење начина на који сервер додељује испоруку, израчунава руту или обрађује локацију. На крају су приказани резултати провере локалног A* поступка и ограничења модела која утичу на тумачење добијене процене времена.

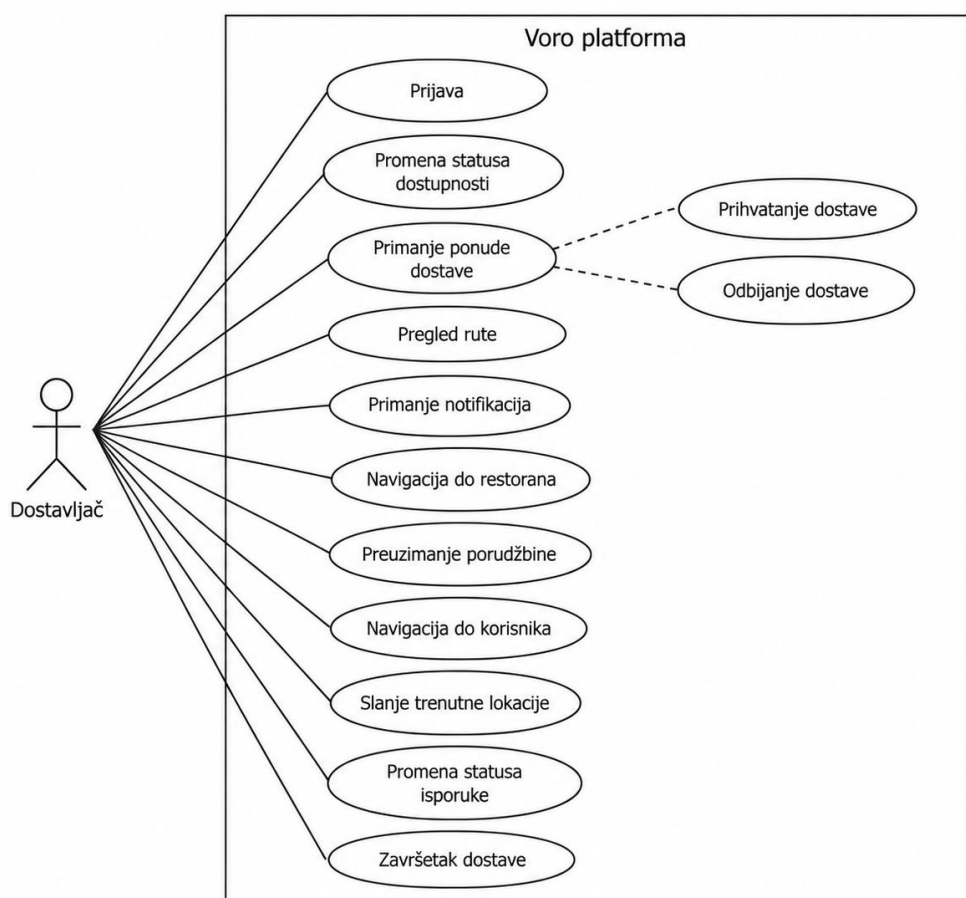
2. АРХИТЕКТУРА СИСТЕМА

2.1. УЧЕСНИЦИ И ОДГОВОРНОСТИ

Систем има четири групе корисника: купце, ресторане, достављаче и администраторе. Купац приступа каталогу, бира производе, задаје адресу и прати своју поруџбину. Ресторан одржава понуду и управља припремом. Достављач пријављује доступност и локацију, разматра понуде и потврђује преузимање и испоруку. Администратор надзире податке и оперативне проблеме. За сваку групу постоји засебна клијентска апликација, али све користе заједнички сервер и заједнички модел података.

Одговорност за пословну исправност налази се на серверу. Изабрана адреса мора припадати купцу; производи морају припадати одговарајућем ресторану; прихваћена понуда мора бити намењена пријављеном достављачу. Приказ дугмета у апликацији није довољна контрола. Сервер проверава идентитет, власништво над ресурсом и тренутно стање пре него што дозволи измену. Тако исти услови важе и када се захтев пошаље непосредно преко програмског интерфејса.

На слици 2.1 приказане су функционалне могућности достављача. Дијаграм повезује доступност, понуду, руту, локацију и потврду испоруке. Оне припадају истом пословном процесу, али се извршавају различитим серверским операцијама. Одбијање понуде, на пример, мења њен статус, док слање локације ажурира оперативни положај и не представља прихватање испоруке.

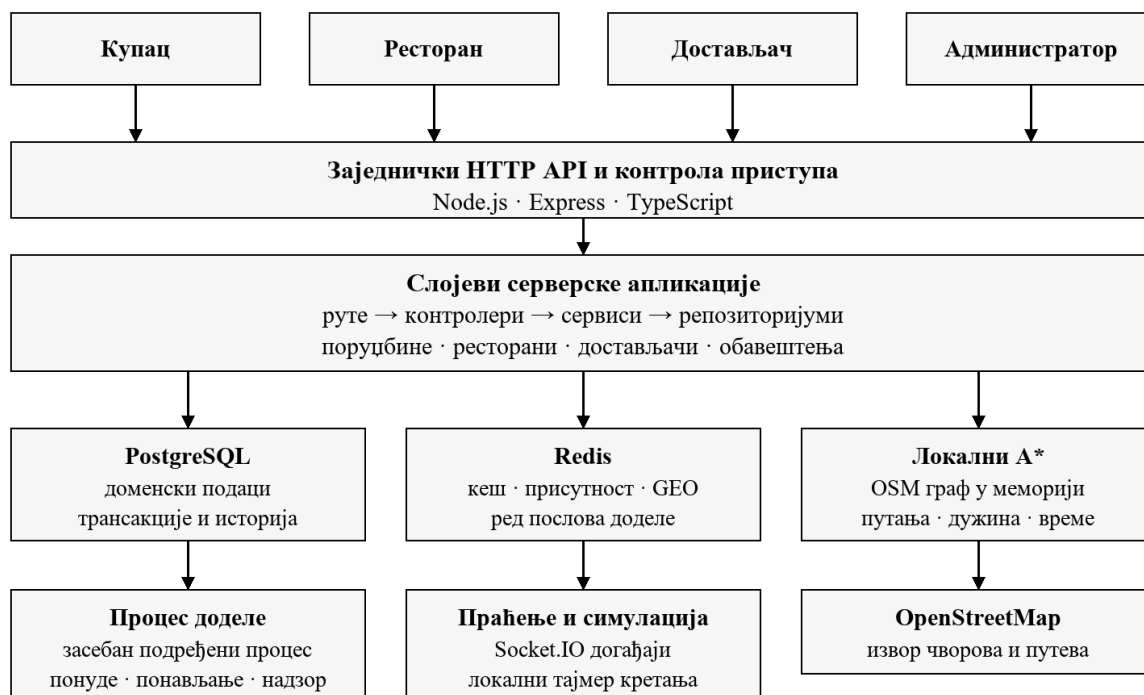


Слика 2.1. Функционалне могућности достављача.

2.2. МОДУЛАРНА И СЛОЈЕВИТА ОРГАНИЗАЦИЈА

Voro је модуларни монолит. Сервер је једна апликација заснована на окружењу Node.js, библиотеци Express и језику TypeScript. Домен је подељен на модуле за аутентификацију, купце, ресторане, достављаче, администрацију, доделу, рутирање и обавештења. Модули деле конфигурацију и инфраструктуру. Назив „сервис“ у коду означава групу функција пословне логике; не подразумева посебно распоређен мрежни микросервис.

Логичка организација представљена је на слици 2.2. HTTP интерфејс, позадинска додела и географска обрада припадају серверском систему. PostgreSQL чува трајно пословно стање. Redis убрзава приступ привременим подацима, проналажење блиских достављача и преузимање задатака за доделу. OpenStreetMap обезбеђује изворне податке о путној мрежи. Избор пута из тих података извршава локална A* имплементација.



Модуларни монолит са пратећим процесом доделе

Слика 2.2. Архитектура серверског система и његове зависности.

Слојеви обраде су рута, контролер, сервис и репозиторијум. Рута повезује HTTP метод и путању са одговарајућим контролером. Посредничке функције проверавају приступ и ограничења захтева. Контролер издваја параметре, позива сервис и обликује HTTP одговор. Сервис примењује пословна правила, а репозиторијум извршава параметризоване SQL упите. Оваква расподела смањује мешање протокола, доменске логике и начина чувања података.

Изоловање рутирања у модуле routingService и aStarRouting омогућава да корисници услуге добију стабилан резултат без познавања XML формата географских података. Сервис достављача само прослеђује полазиште и одредиште. Алгоритам затим брине о учитавању графа, провери приступа путевима, проналажењу најближих чворова и претрази. Тај интерфејс користи и демонстрациони процес, па су координате кретања усклађене са израчунавом рутом.

2.3. ПРОЦЕСИ И КОМУНИКАЦИЈА

Улазна тачка сервера покреће HTTP сервер и Socket.IO. Након успостављања везе са системом Redis отвара се серверски порт и покрећу се позадинске активности. Процес за доделу достављача покреће се као засебан подређени процес. Родитељ надзире његову спремност и при неочекиваном прекиду заказује поновно покретање. Почетно чекање је једна секунда и експоненцијално расте до највише 30 секунди. То смањује број узастопних покушаја када грешка траје.

Локални демонстрациони процес има другачију организацију. Он се извршава као периодични тајмер у основном серверском процесу. Његова улога је померање симулираног достављача дуж већ израчунате A* линије. Додела и симулација зато нису исти механизам: прва управља понудама и избором извршиоца, док друга производи координате за демонстрацију активне испоруке.

Клијентске измене првенствено долазе преко HTTP интерфејса. Socket.IO служи за слање новог стања заинтересованим клијентима. Порука преко тог канала не замењује трансакцију у бази. Прво се потврђује пословна промена, а затим се објављује догађај. Ако клијент прекине везу, стање се може поново прочитати HTTP захтевом. Ова веза између трајног записа и краткотрајног обавештења важна је за опоравак након прекида мреже.

3. ОРГАНИЗАЦИЈА BACKEND-А И ОБРАДА ЗАХТЕВА

3.1. УЛАЗНИ HTTP СЛОЈ

Express усмерава захтев према комбинацији метода и путање [1]. У Voro систему главни простори путања су /auth, /customer, /restaurant, /driver и /admin. Раздвајање прати улоге, док ресурси унутар њих представљају поруџбине, понуде, испоруке, адресе и обавештења. Ова организација олакшава примену контроле приступа на читаву групу операција.

HTTP интерфејс користи JSON тела и статусне кодове. Идеја ресурса и стандардних метода одговара уобичајеној REST организацији комуникације [2]. Ипак, поједине доменске акције имају именоване путање, као што су прихватање понуде и повлачење са доставе. То је практичан интерфејс за конкретан пословни процес, а не тврдња да су доказана сва формална ограничења REST архитектонског стила.

Пре обраде примењују се CORS правила, безбедносна HTTP заглавља, ограничење JSON тела на 100 kB и провере захтева. Дозвољени извори зависе од конфигурације, а развојно окружење дозвољава и одређене локалне адресе. Дневник захтева бележи путању без упитног дела, чиме се избегава уобичајено откривање података из претраге адреса и једнократних параметара пријаве.

3.2. АУТЕНТИФИКАЦИЈА И ОВЛАШЋЕЊА

Клијент доказује идентитет приступним токеном. Посредничка функција проверава токен и допуњава контекст захтева подацима о кориснику. За достављачке операције додатно се захтева улога courier. Идентификатор достављача добија се из пријављеног корисника, а не преузима се као произвољно поље тела. Тиме се спречава да промена једног идентификатора омогући управљање туђом доставом.

Ограничења учесталости издвојена су према врсти операције. Достављачке измене имају квоту од 80 захтева у десет минута, поруке 40, а слање присутности 900. Разлог за већу квоту локације је њена периодична природа. Када би све операције делиле мало ограничење, редовно праћење би могло да онемогући прихватање понуде или потврду преузимања. Купчево креирање поруџбина и сродне измене користе посебно ограничење од 20 захтева у десет минута.

Поред улоге, проверава се приступ конкретном ресурсу. Купац може читати руту сопствене поруџбине, а достављач руту додељене активне испоруке. Слична провера постоји за приступ порукама и Socket.IO соби. Без ње би исправно потписан токен доказао само идентитет, али не и право да се читају туђи подаци. Овлашћење је зато комбинација улоге, власништва и текућег стања.

3.3. КРЕИРАЊЕ ПОРУЏБИНЕ

Креирање почиње захтевом POST /customer/orders. Контролер узима кориснички идентитет, тело и, ако постоји, заглавље Idempotency-Key. Сервис нормализује податке, проверава постојање и активност ресторана, прихватање поруџбина и радно време. Потом бира адресу која припада купцу. Када идентификатор адресе није наведен, користи се подразумевана или прва доступна адреса. Адреса и ресторан морају имати географске координате.

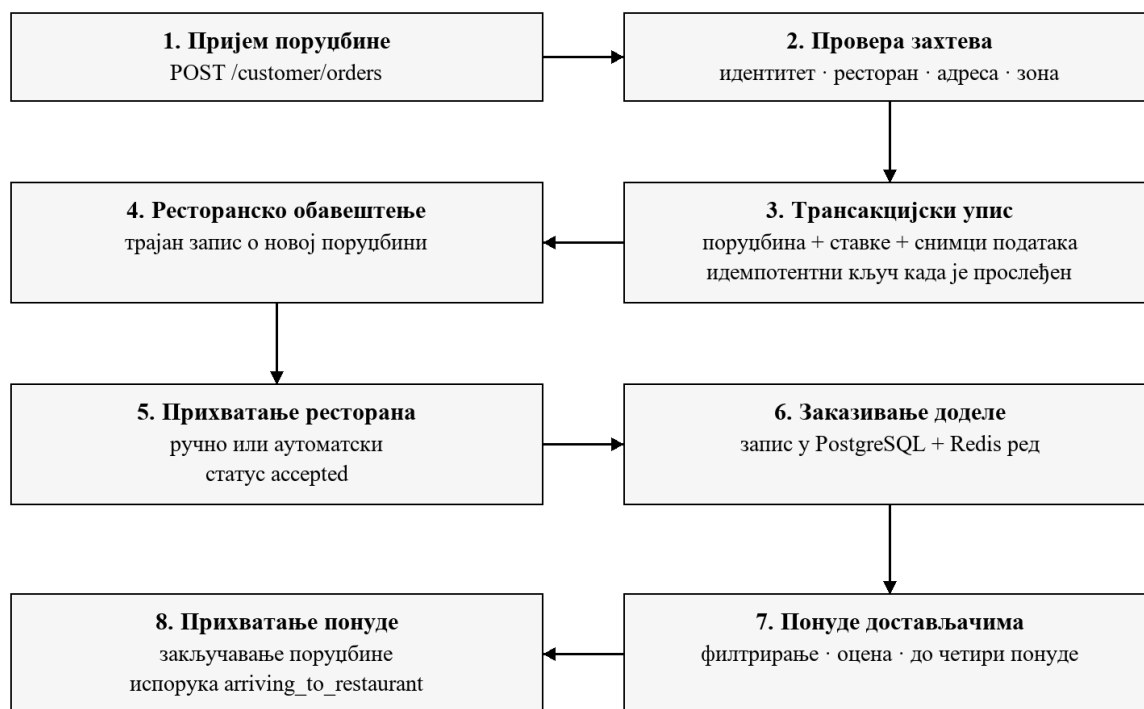
Подручје испоруке проверава се ваздушним растојањем између ресторана и адресе. Ако растојање прелази конфигурисани полупречник ресторана, поруџбина се одбија. Ова провера представља грубу пословну зону испоруке. Она не израчунава

Портал за доставу хране са применом алгоритма A* за оптимизовање руте дужину вожње и не доказује постојање проходне улице. Детаљно рутирање припада каснијем кораку у коме је потребна путна линија.

Репозиторијум проверава производе и креира поруџбину са ставкама. Цене и доступност преузимају се из серверских података. У ставке се уписује тадашњи назив производа и цена, а у поруџбину снимак адресе и координата. Тако каснија промена менија или корисничке адресе не мења смисао већ прихваћене поруџбине. Почетни статус је pending, осим када ресторан користи аутоматско прихватање.

Идемпотентни кључ спречава да поновљени захтев са истим садржајем произведе другу поруџбину. Дозвољени су кључеви од 16 до 128 знакова из ограниченог скупа. Уз кључ се чува отисак нормализованог садржаја. Ово је важно ако је сервер већ извршио упис, а купац није добио одговор због прекида везе. Поновни покушај тада треба да врати постојећи исход. Кључ је подржан, али није обавезан, па његово изостављање не обезбеђује исту заштиту од дуплирања.

После новог уписа креира се ресторанско обавештење. Ако је укључено аутоматско прихватање, статус се поставља на accepted и задатак се прослеђује механизму доделе. Код ручног прихватања то се дешава тек када ресторан потврди поруџбину. Додела може почети током припреме, што омогућава да достављач крене ка ресторану пре завршетка хране. Слика 3.1 приказује овај ток и место трансакције.



Припрема хране и долазак достављача могу се одвијати паралелно.

Слика 3.1. Креирање поруџбине и покретање доделе након прихватања.

3.4. ЖИВОТНИ ЦИКЛУСИ И СЕРВЕРСКИ ИНТЕРФЕЈСИ

Поруџбина, испорука, задатак доделе и понуда имају одвојена стања. Поруџбина описује припрему и исход куповине. Испорука описује транспорт. Задатак доделе памти покушаје проналажења извршиоца, док понуда бележи одговор једног достављача.

Табела 3.1 приказује основне скупове вредности. Њихово раздвајање спречава да истек једне понуде буде погрешно протумачен као отказивање поруџбине.

Табела 3.1. Основна стања пословних и оперативних записа.

Запис	Вредности статуса
Поруџбина	pending, accepted, preparing, ready, picked_up, delivered, cancelled
Испорука	assigned, arriving_to_restaurant, picked_up, on_the_way, delivered, failed, cancelled
Задатак доделе	queued, matching, waiting_for_driver, assigned, failed
Понуда	pending, accepted, declined, expired

Ресторан дозвољава прелазе pending у accepted или cancelled, accepted у preparing или cancelled и preparing у ready или cancelled. Купац може сам отказати поруџбину док је pending. Код преузимања се захтева да ресторан већ означи храну као ready. Достављач затим пролази кроз picked_up, on_the_way и delivered. Додељивање није посебан статус поруџбине, а стање on_the_way припада испоруци. Зато се ознаке приказане кориснику не смеју произвољно пресликати у непостојеће вредности базе.

Табела 3.2 повезује најважније интерфејсе са њиховом улогом. Одговор руте користи тренутну локацију и циљ који зависи од фазе испоруке. Промена статуса проверава власништво, дозвољени прелаз и, када је потребно, физичку близину одредишта. Руте за читање и руте за измену зато не представљају исту операцију чак ни када се односе на исти идентификатор.

Табела 3.2. Кључни серверски интерфејси за извршење испоруке.

Метод и путања	Намена
POST /customer/orders	Креирање поруџбине
PATCH /restaurant/orders/:orderId/status	Прихватање и припрема
POST /driver/offers/:offerId/accept	Трансакцијско прихватање понуде
GET /driver/offers/:offerId/route-estimate	Процена обе деонице понуде
GET /driver/deliveries/:deliveryId/route	A* рута до текућег одредишта
PATCH /driver/presence	Доступност и узорак локације
PATCH /driver/deliveries/:deliveryId/status	Преузимање, полазак и испорука
GET /customer/orders/:orderId/route	Рута активне испоруке купцу

4. ПОДАЦИ И ИНФРАСТРУКТУРА

4.1. РЕЛАЦИОНИ МОДЕЛ

PostgreSQL чува доменске записе и њихове односе. Корисник је повезан са улогом, адресама и подацима налога. Ресторан је повезан са категоријама и производима. Поруџбина повезује купца, ресторан и ставке, док испорука повезује поруџбину са достављачем. Идентификатори главних табела су целобројни, а везе се реализују страним кључевима. Модел се развија нумерисаним SQL миграцијама, тако да нова инсталација добија структуру у одређеном редоследу.

Постојање одвојених табела за поруџбину и ставке омогућава произвољан број производа без понављања података купца. Одвојена испорука чува транспортне временске ознаке и достављача, а може се поново доделити ако претходни покушај пре преузимања не успе. Оперативне понуде смештене су у `delivery_dispatch_offer`, а целокупан поступак доделе у `delivery_dispatch_job`. Један задатак може имати више понуда, укључујући понуде из различитих покушаја.

Табела 4.1 издваја групе записа које су најважније за backend. Географски граф за A* није део приказаног релационог модела. Он се гради из спољних података и привремено држи у меморији процеса. Насупрот томе, историја стварне или симулиране локације испоруке чува се у бази и служи праћењу и анализи конкретног пословног догађаја.

Табела 4.1. Улоге главних група табела.

Група	Представници	Улога
Идентитет	user, role, refresh_tokens	Налози, улоге и сесије
Каталог	restaurant, product, product_category	Ресторани и понуда
Куповина	order, order_item, address	Поруџбина, ставке и адреса
Транспорт	courier, delivery, delivery_status	Извршилац и ток испоруке
Додела	delivery_dispatch_job, delivery_dispatch_offer	Покушаји и понуде
Праћење	delivery_location	Локацијски узорци и квалитет

4.2. ТРАНСАКЦИЈЕ И КОНКУРЕНТНИ ЗАХТЕВИ

Трансакција повезује више уписа у јединствену целину: потврђује се цео скуп измена или се измене поништавају [3]. У Voro систему то је посебно важно при прихватању понуде, јер више достављача може готово истовремено покушати да прихвати исту поруџбину. Није довољно претходно прочитати стање у апликацији, пошто други захтев може да га промени пре уписа.

Репозиторијум закључава изабрану понуду, проверава рок, затим узима трансакцијско саветодавно закључавање по идентификатору поруџбине. Проверава се и

Портал за доставу хране са применом алгоритма A* за оптимизовање руте доступност достављача, постојећа испорука и статус поруџбине. При успешном прихватању понуда постаје accepted, друге понуде за исту поруџбину истичу, а испорука се креира или поново додељује са статусом arriving_to_restaurant. Задатак доделе добија assigned. Цео исход се потврђује као повезана промена.

На тај начин победник конкурентних захтева одређује се у бази. Клијент који стигне касније добија информацију да понуда више није доступна. Сличан принцип важи за промене статуса: текући запис се чита под закључавањем и проверава дозвољени следећи корак. Ово одржава исправност и када клијент понови захтев или корисник ради са више отворених прозора.

4.3. УЛОГА СИСТЕМА REDIS

Redis се користи за привремене податке и брзе оперативне упите. Кеш каталога смањује поновљено читање ресторана и менија. Бројачи учесталости штите интерфејсе, а ред за доделу омогућава да позадински процес брзо преузме нови посао. Геопросторни индекс чува координате достављача и омогућава претрагу околине одређене тачке, што је типична намена геопросторних структура система Redis [4].

Запис присутности има рок од 50 секунди. После истека се више не третира као свеж Redis сигнал. PostgreSQL провера кандидата прихвата локације из последњих 120 секунди, па се извори комбинују. Redis подаци могу дати новију позицију, док база задржава податке о налогу и испоруци. Кратак рок присутности омогућава да прекинут клијент временом престане да изгледа активно чак и ако није послао уредну одјаву.

Сесије налога и токени за освежавање припадају трајном моделу у бази. Кеш путних графова припада меморији A* модула. Зато је важно разликовати ове механизме од Redis кеша. Њихови рокови и домети су различити: истек графа захтева поновно учитавање путева, док истек присутности утиче на могућност избора достављача. Јединствен назив „кеш“ не значи да су подаци смештени на истом месту.

5. АЛГОРИТАМ ПРОНАЛАЖЕЊА ДОСТАВЉАЧА

5.1. ПОКРЕТАЊЕ И ИЗБОР КАНДИДАТА

Додела се активира након прихватања поруџбине. Функција `enqueueDispatch` уписује задатак у базу и прослеђује идентификатор у Redis ред. Позадински циклус ради на 1,5 секунди, а на 20 секунди проверава и базу за послове који чекају. Двоструки извор смањује зависност од једног краткотрајног сигнала. Обрада захтева да поруџбина буде `accepted`, `preparing` или `ready` и да нема активну испоруку, односно да претходна има завршно стање које дозвољава нови покушај.

Кандидати се прикупљају из Redis геопросторне претраге и базе. База тражи достављача који је на мрежи, доступан, има координате и свежу локацију. Резултати се спајају по идентификатору, а Redis координате замењују старије координате истог кандидата. Систем тиме не одбацује све кандидате из базе само зато што је пронађен један Redis запис. Потом се читају број активних испорука и број још важећих понуда сваког кандидата.

Полупречник претраге односи се на пут достављача до ресторана. Код га изводи из ресторанског полупречника испоруке множењем вредности у километрима са 750 и заокруживањем. Резултат се ограничава на најмање 3.000 и највише 6.000 m; помоћна подразумевана вредност у сервису је 5.000 m. То није јединичка конверзија километара у метре, већ пословни коефицијент којим се подручје преузимања сужава. За ресторански полупречник од 8 km добија се горња граница од 6 km.

5.2. РАСТОЈАЊЕ И ОЦЕНА КАНДИДАТА

За растојање се користи Хаверсинова формула, са полупречником Земље $R = 6.371.000$ m. Географске ширине φ и дужине λ најпре се изражавају у радијанима. Помоћна величина a и растојање d одређени су једначинама 5.1 и 5.2. Резултат представља удаљеност по сферној површини, без утицаја улица, смерова кретања и препрека.

$$a = \sin^2\left(\frac{\Delta\varphi}{2}\right) + \cos \varphi_1 \cdot \cos \varphi_2 \cdot \sin^2\left(\frac{\Delta\lambda}{2}\right) \quad (5.1)$$

$$d = 2R \cdot \operatorname{atan2}(\sqrt{a}, \sqrt{1-a}) \quad (5.2)$$

Кандидати ван полупречника и они који већ имају најмање две отворене понуде избацују се из разматрања. Преостали се сортирају по оцени S из једначине 5.3. Ознака L представља број активних испорука, а P број важећих понуда. Нижи резултат има предност. Казнени коефицијенти имају бројчани ефекат еквивалентан додатној удаљености у метрима, али нису физички пут који достављач треба да пређе.

$$S = d + 2000L + 2500P \quad (5.3)$$

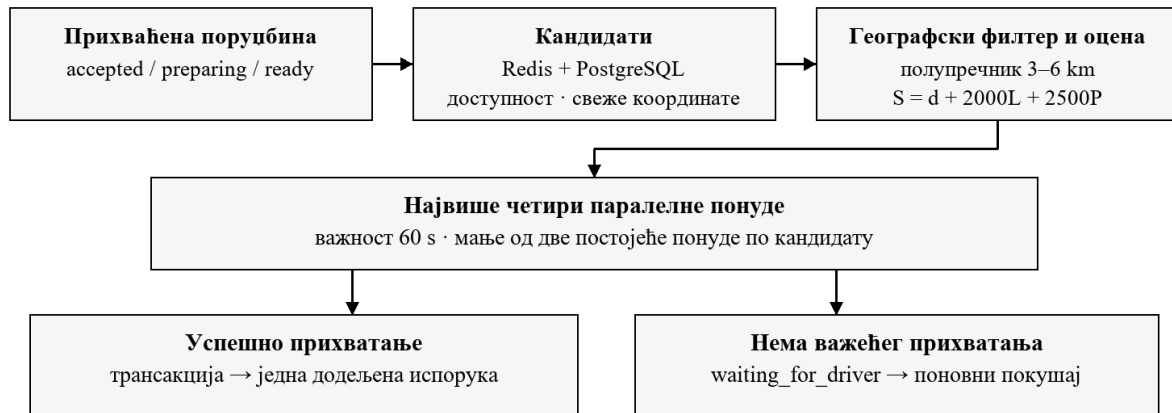
Пример у табели 5.1 показује зашто најближи достављач не мора бити први у рангу. Достављач А је ближи ресторану, али већ има отворену понуду. Достављач Б је даљи и неоптерећен, па добија мању оцену. Бројеви су рачунски пример формуле, а не мерење стварних корисника. Сврха примера је да покаже како систем распоређује понуде и избегава стално оптерећивање истог најближег кандидата.

Кандидат	d y m	L	P	Оцена S
A	800	0	1	3300
Б	1500	0	0	1500
B	1100	1	0	3100

5.3. ПОНУДЕ И ПОНОВНИ ПОКУШАЈИ

У једном кругу бирају се највише четири кандидата и шаљу им се паралелне понуде. Понуда важи 60 секунди. Први важећи захтев који успешно прође трансакцијску проверу добија испоруку. То се разликује од строго секвенцијалног приступа у коме се чека одговор првог кандидата пре контактирања другог. Паралелна рунда скраћује чекање, док база обезбеђује јединствен исход.

Када нема одговарајућих кандидата, задатак добија `waiting_for_driver` и нови покушај после 30 секунди. Када су понуде послате, следећи покушај се заказује у складу са њиховим прозором важења. Сервис памти број покушаја. За одређене неуспешне случајеве подиже упозорење након четири покушаја, а критично упозорење након осам. То је оперативни сигнал за надзор, а не доказ да су сви достављачи одбили поруџбину. Слика 5.1 обједињује филтрирање, рангирање и исход рунде.



Без кандидата: нови покушај после 30 s.

A* се користи за руту и време; ова оцена користи Хаверсиново растојање.

Слика 5.1. Ток филтрирања, рангирања и понуде испоруке.

За N кандидата израчунавање растојања и оцена је линеарно, а сортирање захтева ред величине $N \log N$ операција. У овом кораку се не покреће A* за сваког кандидата. То чува ресурсе, али значи да река, једносмерна улица или непостојећа веза могу учинити да ближи кандидат има дужу стварну вожњу. Могуће унапређење је детаљно рутирање само за ужи скуп најбоље ранжираних кандидата. Садашња реализација задржава једноставну географску оцену.

6. A* АЛГОРИТАМ И ФОРМИРАЊЕ РУТЕ

6.1. МОДЕЛ ПУТНЕ МРЕЖЕ

Путна мрежа представља се усмереним тежинским графом $G = (V, E)$. Скуп V садржи географске чворове, а скуп E дозвољене везе између суседних чворова. Чвор чува идентификатор, географску ширину и дужину, као и листу излазних грана. Грана садржи одредишни чвор, растојање у метрима и трошак. У Voro имплементацији трошак је процењено време проласка у секундама. Зато A* тражи најбржу руту према усвојеним брзинама, која не мора имати најмању дужину.

OSM чворови нису само раскрснице. Могу представљати и тачке геометрије кривудае улице. Узастопни чворови једног путног објекта повезују се гранама, тако да линија добијена реконструкцијом прати облик улице. Када би се користиле само крајње координате, линија би могла да пресеца блокове или да изгуби стварну дужину кривине. Усмереност је важна јер постојање гране од u до v не подразумева дозвољен повратни смер.

OpenStreetMap у овом решењу обезбеђује податке о мрежи. Захтев за просторни исечак враћа XML елементе чворова и путева са њиховим ознакама. Изворни API подржава избор подручја преко ограничавајућег правоугаоника [5]. Voro затим локално тумачи податке и бира путању. У актуелном извршном ланцу `routingService` позива `findAStarDrivingRoute`; не позива се OSRM сервис за готову руту. Разлика између извора географских података и алгоритма који их претражује пресудна је за разумевање архитектуре.

6.2. ИЗБОР ПОДРУЧЈА И УЧИТАВАЊЕ ГРАФА

Пре рутирања проверавају се коначност бројева и дозвољени опсези координата. Географска ширина мора бити између -90 и 90 степени, а дужина између -180 и 180 . Ваздушно растојање полазишта и циља ограничено је на 30 km. То је ограничење величине захтева за ову примену, а не максимална дужина добијене путне линије. Обилазна возња може бити дужа од ваздушног растојања.

Основни правоугаоник обухвата обе тачке са додатном маргином. Маргина ширине је најмање $0,003$ степена или 20% разлике ширина, а маргина дужине најмање $0,004$ степена или 20% разлике дужина. Тако претрага добија простор за суседне улице и краће обиласке. Ако се у почетном исечку не пронађе веза, покушавају се правоугаоници проширени факторима $1,6$ и $2,25$. Геометријско проширење није промена алгоритма A*, већ покушај да се обезбеди шири скуп дозвољених путева.

Захтев за податке има подразумевани рок од 25 секунди, подесив конфигурацијом. Када извор врати одговор 400 , имплементација га третира као ограничење величине или густине исечка и дели правоугаоник на четири дела. Дељење се може поновити до три нивоа. Појединачни делови се учитавају редом, а графови се спајају према идентификаторима чворова. Дуплиране гране са истим одредиштем и трошком не додају се поново. Један одговор 400 не доказује сам по себи густину података; то је начин на који га овај код обрађује.

У меморији процеса чува се највише шест графова, сваки са роком од 20 минута. Постојећи граф може послужити ако његов правоугаоник потпуно покрива тражено подручје. Употребљени запис помера се на почетак списка, а најстарије коришћен запис испада при прекорачењу капацитета. Рок се рачуна од смештања и не продужава се

Портал за доставу хране са применом алгоритма A* за оптимизовање руте сваким читањем. Зато поновљена рута у блиском подручју обично не захтева ново преузимање целе мреже.

Кеш припада једном серверском процесу. Поновно покретање га празни, а више серверских инстанци не дели аутоматски исти садржај. Два истовремена захтева за празан кеш могу независно покренути преузимање, јер нема посебног обједињавања захтева који су већ у току. Ове особине не мењају логички поступак A*, али утичу на време одговора и оптерећење спољног извора.

6.3. ПРОХОДНОСТ, СМЕР И БРЗИНА

При изградњи графа разматрају се путеви са ознаком highway. Избацују се пешачке и бициклистичке стазе, степенице, планирани и путеви у изградњи, као и друге наведене непогодне категорије. Објекат означен као површина не користи се као обична улица. Ознаке access, vehicle и motor_vehicle са вредношћу по или private такође доводе до искључивања. Филтрирање је приближни модел кретања моторног возила, а не универзални модел за све типове достављача.

Ознака oneway са вредностима yes, true или 1 формира гране само у смеру редоследа чворова. Вредност -1 обрће смер. Кружни ток се такође третира као једносмеран унапред. Остали случајеви добијају обе гране. Имплементација не тумачи све могуће OSM изузетке; на пример, правило за кружни ток има предност над уобичајеним двосмерним случајем. Забране скретања из OSM релација нису део овог модела графа.

Брзина се најпре чита из maxspeed:forward или maxspeed:backward, у зависности од смера. Затим се користи општа maxspeed ознака, а ако нема употребљивог броја, подразумевана вредност за категорију пута. Ознака mph претвара се у километре на час. Бројчане вредности ограничавају се на опсег 5–130 km/h. Парсер узима први број који препозна, па сложена условна ограничења не представљају потпуно моделована правила.

Табела 6.1 показује неке подразумеване брзине. Оне су параметри прорачуна, а не измерене просечне брзине стварних достављача. За моторни пут користи се 100 km/h, за примарни 60, секундарни 50, а за стамбену улицу 30. Приступни пут је спорији, као и улица са смиреним саобраћајем. Ове разлике омогућавају да дужи, али бржи правац има мањи трошак од кратке споре улице.

Табела 6.1. Изабране подразумеване брзине у моделу рутирања.

OSM категорија	Брзина у km/h
motorway	100
trunk	80
primary	60
secondary	50
tertiary	45
residential	30

OSM категорија	Брзина у km/h
service	15
living_street	10

6.4. ТРОШАК ПУТА И ХЕУРИСТИКА

Основу A* поступка чини комбиновање већ познатог трошка и процене преосталог трошка. Hart, Nilsson и Raphael формализовали су овакав приступ претрази пута минималне цене [6]. У једначини 6.1, $g(n)$ је најмањи до тада пронађени трошак од полазишта до чвора n , $h(n)$ је процена до циља, а $f(n)$ је приоритет за избор следећег чвора.

$$f(n) = g(n) + h(n) \quad (6.1)$$

У Voro графу трошак гране од u до v рачуна се као растојање $d(u,v)$ подељено брзином $v(u,v)$, израженом у метрима у секунди. Збир грана зато има јединицу секунде. Хеуристика је Хаверсиново растојање од текућег чвора до циљног чвора подељено са 130 km/h, односно приближно 36,11 m/s. То је фиксна горња брзина модела, а не максимум накнадно измерен у конкретном графу.

$$c(u, v) = \frac{d(u,v)}{v(u,v)} \quad (6.2)$$

$$h(n) = \frac{d(n, \text{циљ})}{\frac{130}{3,6}} \quad (6.3)$$

За гране настале из овог модела хеуристика не прецењује време: пут по мрежи није краћи од географског растојања, а ниједна усвојена брзина не прелази горњу границу. Иста веза са неједнакошћу троугла даје конзистентност процене унутар изграђеног графа. Због тога је хеуристика безбедна за тражење минималног моделованог времена. Она је намерно оптимистична и у спорој градској мрежи може бити слабије усмеравајућа, јер претпоставља много већу брзину од стварно додељене већини улица.

6.5. СТРУКТУРЕ ПОДАТАКА И КОРАЦИ ПРЕТРАГЕ

Отворени чворови смештени су у минимални приоритетни ред реализован бинарном хипом. Убацавање подиже елемент док родитељ нема мањи или једнак приоритет. Уклањање узима корен, премешта последњи елемент и спушта га до исправног места. Основне операције зато зависе логаритамски од броја елемената. Ово је ефикасније од претраживања целе листе ради проналажења најмањег f при сваком кораку.

Мапа costs чува најбољи познати g , а sameFrom претходника сваког достигнутог чвора. Скуп closed бележи обрађене чворове. Код у ред додаје нову ставку када се пронађе бољи пут, уместо да непосредно смањује приоритет већ постојеће ставке. Због тога ред може садржати више ставки истог чвора. При уклањању се застарела ставка прескаче ако њен упамћени трошак више не одговара мапи costs. Прескаче се и ставка чвора који је већ затворен без новог побољшања.

Ако се преко текућег чвора пронађе јефтинији пут до суседа, мењају се g и претходник, сусед се уклања из затвореног скупа и поново додаје у ред. Ово омогућава

Портал за доставу хране са применом алгоритма A^* за оптимизовање руте поновно отварање. Негативни и неконачни трошкови гране се игноришу. У путном графу нормалне гране имају позитивну дужину и брзину. Алгоритам се завршава када се циљ уклони као важећа најбоља ставка или када ред остане празан.

Псеудокод 6.1 приказује поступак без детаља XML обраде. Реконструкција креће од циља, прати мапу претходника до полазишта и обрће редослед. Тај низ чворова је основа полилиније. Ако су почетни и циљни чвор исти, језгро враћа један чвор и нулти трошак, након чега спољни поступак и даље обрађује приступне деонице.

Псеудокод 6.1. Претрага графа алгоритмом A^* .

```
g[почетак] ← 0; орен.додај(почетак, h(почетак))
док орен није празан:
    текући ← орен.извадиНајмањиПриоритет()
    ако је ставка застарела или чвор затворен: настави
    ако је текући = циљ: врати реконструисан пут и g[циљ]
    затвори текући
    за сваку дозвољену грану текући → сусед:
        новиТрошак ← g[текући] + ценаГране
        ако је новиТрошак мањи од познатог g[сусед]:
            претходник[сусед] ← текући
            g[сусед] ← новиТрошак
            поново отвори сусед ако је затворен
            орен.додај(сусед, g[сусед] + h(сусед))
врати да пут није пронађен
```

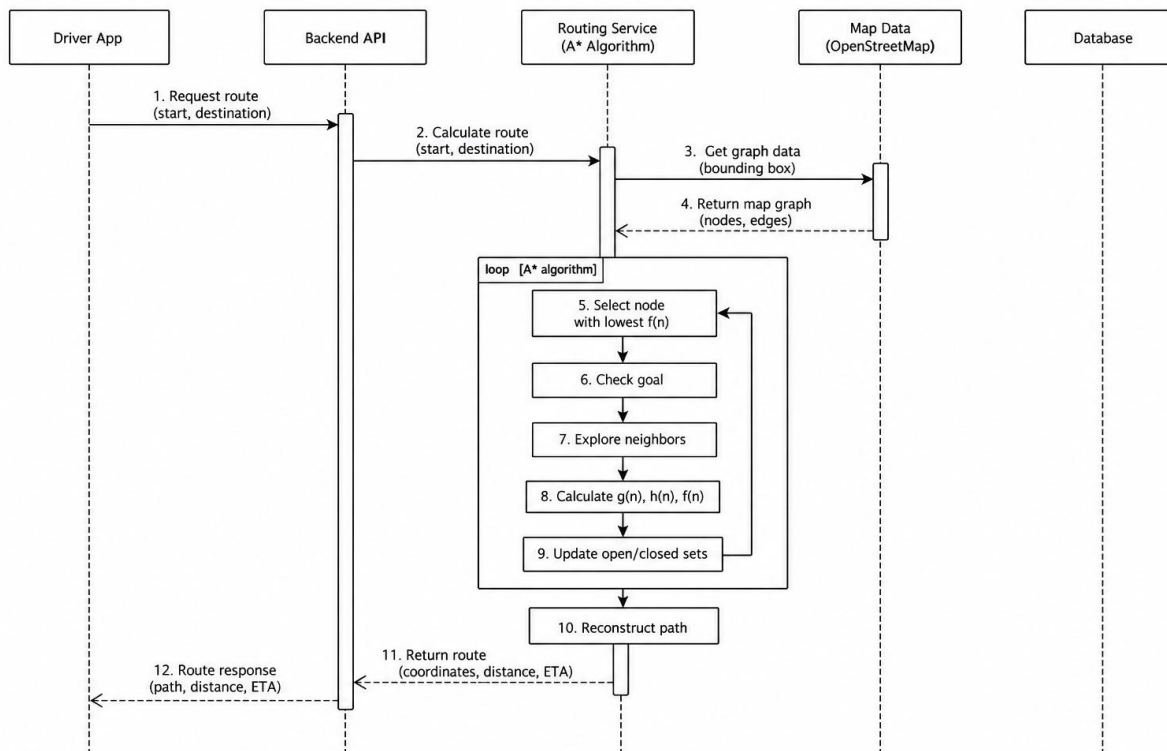
Када је хеуристика конзистентна и сваки чвор се ефикасно обради једном, уобичајена процена рада са бинарном хипом је реда $(V + E) \log V$, уз простор за граф и помоћне структуре. Реализација са дуплираним ставкама захтева и простор за застареле уносе у ред. Генерички поступак дозвољава поновно отварање, па се једноставна граница не сме безусловно приписати свакој произвољној хеуристици. Код стварног сервиса у време одговора улазе и мрежно преузимање, XML обрада и проналажење приступних чворова.

6.6. ПОВЕЗИВАЊЕ КООРДИНАТА И РЕЗУЛТАТ

Полазиште и циљ обично нису тачно на OSM чворовима. За почетак се бира најближи чвор са излазном граном, а за циљ најближи чвор који има улазну везу. Претрага најближег чвора пролази кроз граф. Ако је најближа погодна тачка удаљена више од 400 m, рута се одбија. Поступак бира чвор, а не пројекцију на средину сегмента улице, што поједностављује модел, али може утицати на приступну дужину.

Резултујућа линија садржи стварно полазиште, чворове пронађеног пута и стварни циљ. Скоро подударне узастопне тачке, ближе од 0,2 m, изостављају се. Координате се враћају у редоследу [географска ширина, географска дужина]. Почетни и завршни спој до графа моделирани су правом линијом и брзином од 20 km/h. То су приступни сегменти, а не засебно проверене уличне гране. Могу проћи кроз географску препреку коју ово поједностављење не тумачи.

На слици 6.1 приказана је комуникација приликом добијања руте. Извор мапе обезбеђује податке, док Routing Service извршава претрагу, реконструише путању и враћа растојање и време. Називи у дијаграму представљају логичке улоге. Претраживање не уписује путни граф као доменску табелу у PostgreSQL.



Слика 6.1. Секвенца локалног A* рутирања над OSM подацима.

Оптималност се односи на пут између изабраних чворова у учитаном графу, са усвојеним тежинама. Прва успешна величина исечка прекида проширивање. Зато постојање пута у уском исечку не доказује да изван њега нема бржег великог обиласка. Такође, најближи приступни чвор не мора дати најбољу комбинацију приступа и уличне вожње. Ова ограничења одређују домет резултата и важна су за практичну употребу речи „оптимална“.

7. ПРОЦЕНА ВРЕМЕНА И ДУЖИНЕ ИСПОРУКЕ

7.1. ВРЕМЕ ВОЖЊЕ ПО A* РУТИ

Време које враћа A* заснива се на збиру времена грана. Ако једна деоница има 300 m и брзину 30 km/h, њен трошак је 36 секунди. Деоница од 600 m при 60 km/h има исти трошак. Овај рачунски пример показује да избор најмањег времена није истоветан избору најмањег броја метара. Дужина и време чувају се као различите величине и објашњавају различите особине пута.

Након претраге додају се времена почетног и завршног приступа. У једначини 7.1, T означава укупан број секунди, g(циљ) трошак уличног пута, а d_p и d_k приступна растојања у метрима. Делилац 20/3,6 претвара брзину приступа из километара на час у метре у секунди. Дужина линије се посебно сабира из враћених координата и заокружује на цео метар.

$$T = g(\text{циљ}) + \frac{d_p + d_k}{\frac{20}{3,6}} \quad (7.1)$$

Број минута добија се заокруживањем навише, уз најмање један минут. Горња граница додаје најмање два минута или 15% основне процене, такође заокружено навише. Једначине 7.2 и 7.3 описују тачно правило. Ознака M представља основну процену у минутима. За T = 310 s добија се M = 6 и опсег 6–8 минута. За M = 20 горња граница је 23 минута.

$$M = \max(1, \lceil \frac{T}{60} \rceil) \quad (7.2)$$

$$ETA = [M, M + \max(2, \lceil 0,15M \rceil)] \quad (7.3)$$

Овај опсег је програмско правило резерве. Није статистички интервал поузданости израчунат из историјских мерења. Модел нема улаз о тренутној гужви, трајању семафорског циклуса, тражењу паркинга или уласку у зграду. Добијена вредност се зато тумачи као процена вожње на основу мреже и брзина. За практично обећање рока испоруке потребно је размотрити и преосталу припрему и друге оперативне застоје.

7.2. ПРОЦЕНА ПОНУДЕ И АКТИВНЕ ИСПОРУКЕ

Пре прихватања понуде достављач може затражити две руте: од тренутног положаја до ресторана и од ресторана до купца. Обе се израчунавају истим A* сервисом. Сервер их покреће паралелно и враћа појединачне резултате, збир дужина, збир процењених минута и збир граница опсега. Ово омогућава процену укупног превоза који понуда подразумева.

Заокруживање се обавља за сваку деоницу пре сабирања. Ако обе трају по 61 секунду, свака добија два минута, па је збир четири минута, иако је заокружен укупан број секунди три минута. То је очекивана последица постојећег интерфејса. Укупна процена понуде такође не додаје чекање на храну. Њено значење је укупан моделовани превоз две деонице, а не потпун рок од тренутка прихватања до предаје.

Код активне испоруке циљ зависи од стања. За assigned и arriving_to_restaurant одредиште је ресторан. За касније активне фазе сервис бира адресу купца. После сваког новог захтева полазиште је тада позната позиција достављача, па је могуће добити преосталу руту. То није аутоматски алгоритам управљања возилом: сервер рачуна податке, док стварно кретање остаје одговорност извршиоца.

Купчев сервис враћа живу руту тек када је испорука *on_the_way* и праћена локација није старија од 45 секунди. У супротном не користи застарелу позицију као привидно актуелно полазиште. Статус поруџбине и подаци о ресторану и адреси могу и даље бити доступни. Разликовање доступности статуса и доступности руте спречава да привремени прекид телеметрије прикрије целокупно стање поруџбине.

7.3. ПРОЦЕНА ПОРУЏБИНЕ ПРЕ ЖИВОГ ПРАЋЕЊА

Листа поруџбина има засебан модел процене. Основни опсег зависи од статуса: *pending* користи 35–45, *accepted* 25–35, *preparing* 15–25, *ready* 10–15, а *picked_up* 5–10 минута. Ове бројеве не производи A* претрага. На њих се додаје допуна према количини производа и припреми ресторана. Табела 7.1 приказује основне вредности, како се не би помешале са временом уличне вожње.

Табела 7.1. Основни статусни опсези пре додатака за припрему.

Статус	Основни опсег у минутима
<i>pending</i>	35–45
<i>accepted</i>	25–35
<i>preparing</i>	15–25
<i>ready</i>	10–15
<i>picked_up</i>	5–10

Допуна за количину је два минута по јединици преко прве, уз максимум од 12 минута. За фазе *pending*, *accepted*, *preparing* и *ready* додаје се већа вредност између подешене припреме ресторана, са подразумеваних 20 минута, и преосталог времена означене заузетости. То је директна формула сервиса. Чак и код *ready* статусни модел још додаје ову компоненту, па његова вредност није истоветна физички преосталој припреми.

На пример, *accepted* поруџбина са три јединице производа, припремом од 20 минута и без додатне заузетости добија $25-35 + 4 + 20$, односно 49–59 минута. Пример показује зашто се статусна процена може значајно разликовати од A* процене од неколико минута. Прва описује конзервативно време на листи поруџбина, а друга вожњу по конкретној рути. Будуће усклађивање модела захтевало би раздвајање преостале припреме, доласка достављача и вожње до купца.

8. ЛОКАЦИЈА И ПРАЋЕЊЕ ИСПОРУКЕ

8.1. ПРИЈЕМ И ПРОВЕРА ЛОКАЦИЈЕ

Достављач шаље присутност преко PATCH /driver/presence. Поред стања на мрежи, порука може садржати координате, тачност, правац, брзину и време снимања. Пријављивање расположивости захтева употребљиве координате. Сервер одвојено проверава да ли је захтев синтаксно исправан и да ли је узорак довољно поуздан за оперативну употребу. Нумерички исправна координата и даље може бити стара или сувише удаљена од претходног положаја.

Пријављена тачност лошија од 80 m и узорак старији од 60 секунди означавају се као неупотребљиви. Када постоји претходна позиција, проверава се дозвољени скок. Граница укључује основну толеранцију од 75 m, допуну за тачност и пређени пут при горњој очекиваној брзини током протеклог времена. Брзина је ограничена на 55 m/s, а достављена брзина може додатно утицати на процену. То је провера веродостојности узорка, не потпуна заштита од лажирања геолокације.

Одбачен узорак не помера оперативну позицију ни индекс за доделу. За активну испоруку може остати забележен као сиров податак са ознаком квалитета. Приказни положај се благо филтрира: нови узорак добија тежину 0,75 при доброј тачности, 0,6 при средњој и 0,45 при слабијој употребљивој тачности. Остатак долази из претходне позиције. Тако се смањује ситно подрхтавање, али се уводи и извесно заостајање приказа за свежим узорком.

За потврду преузимања и завршетка важе строжи оперативни услови. Позиција мора бити свежа у оквиру 45 секунди, а достављач унутар 200 m од ресторана, односно купца. Обавештење о близини купца може се послати унутар 250 m док је испорука on_the_way. Ови прагови служе различитим циљевима. Већи праг најављује долазак, док мањи ограничава потврду радње.

8.2. ОБЈАВЉИВАЊЕ ДОГАЂАЈА

После прихваћене измене сервер синхронизује присутност и објављује догађаје. Socket.IO користи собе user:{id} и order:{orderId}. Соба је серверски механизам за груписање веза и слање поруке одабраним примаоцима [7]. При успостављању везе проверава се JWT приступни токен, а прикључење соби поруџбине захтева проверу права приступа. Само познавање броја поруџбине зато не омогућава читање њене локације.

Догађаји order:tracking, driver:location, order:message и notification:new имају различите намене. Праћење садржи стање испоруке и доступне податке о куриру, док порука разговора садржи комуникациони садржај. Тренутна Socket.IO аутентификација прихвата улоге customer и courier. Не треба из тога извести да ресторан прима исти догађај преко истог канала. Ресторанска обавештења постоје као посебни записи доступни ресторанском интерфејсу.

Локацијски запис и догађај нису исте врсте податка. Запис омогућава касније читање и проверу, а догађај обавештава повезане клијенте о промени. Ако се веза прекине, клијент мора поново прочитати актуелно стање. Слика 8.1 приказује овај ток, укључујући заједничку тачку уласка за стварни уређај и локални демонстрациони извор.



Слика 8.1. Обрада локације и објављивање стања испоруке.

8.3. ЛОКАЛНА СИМУЛАЦИЈА КРЕТАЊА

Демонстрациони процес је омогућен у развојном окружењу осим када је искључен конфигурацијом. Подразумевани интервал је 1.200 ms, а корак 90 m. Он проналази активне испоруке и за сваку памти тренутни положај, путну линију, индекс следеће тачке, одредиште и статус. Ако постоји познат положај достављача, користи га као почетни; иначе полази са померене тачке у близини ресторана.

Пре померања тражи се A* рута. Корак се троши дуж њених сегмената, а када преостали број метара није довољан за цео сегмент, интерполира се међутачка. Ситни сегменти се прескачу. За assigned и arriving_to_restaurant циљ је ресторан. Код picked_up положај се објављује без померања, а код on_the_way циљ је адреса купца. Промена статуса поништава стару руту како би се израчунала одговарајућа нова деоница.

Симулирани положај пролази кроз исти сервис updatePresence као позиција уређаја. Процес привремено ослобађа Redis ознаку контроле симулатора, уписује свој узорак и обнавља ознаку. Она спречава да истовремени GPS сигнал отвореног прегледача стално враћа достављача на другу локацију. Пријављена тачност симулатора је 8 m, а брзина је ограничена на 55 m/s. Ако сервис не прихвати предвиђено померање, симулација наставља од потврђене позиције и поново планира пут.

Параметри 90 m и 1,2 s одговарају убрзаној демонстрацији, а не измереној брзини градске доставе. Када граф није доступан, процес бележи грешку и покушава поново; не замењује уличну руту директном линијом до циља. Симулација сама не потврђује преузимање ни завршетак. Статусне радње и даље пролазе кроз уобичајене серверске провере. Ово омогућава испитивање кретања и праћења без физичког премештања уређаја, уз очуван пословни ток.

9. ПОКРЕТАЊЕ И ПРОВЕРА РЕШЕЊА

9.1. ОРГАНИЗАЦИЈА ОКРУЖЕЊА

Локално окружење описано је конфигурацијом Docker Compose. PostgreSQL 16 и Redis 7 имају сопствене сервисе и трајне волумене. Сервис миграција припрема структуру базе пре покретања backend-а. Сервер и четири клијентске апликације граде се из одвојених контекста, а Caddy усмерава захтеве ка одговарајућој апликацији и API серверу. Ова подела представља начин покретања компонената, док доменска серверска апликација остаје модуларни монолит.

Провера спремности обухвата базу, Redis и процес доделе. Разликовање живости и спремности је практично: процес може да одговара на мрежи иако још не може да обради поруџбину. При прекиду процеса доделе његова спремност се поништава до поновног покретања. За резервне копије постоји посебан профил конфигурације који није подразумевано део сваког локалног покретања.

Доступност географских података остаје спољна зависност чак и када је A* локалан. Сервер може имати здраву базу и исправан алгоритам, али не успети да учита мрежу за ново подручје. Кеш ублажава такву зависност само за већ учитане графове. За интензивну употребу погоднији би био контролисани извор исечака или сопствени припремљени граф, уз поштовање услова коришћења података. То је могућност развоја, а не опис садашње инфраструктуре.

9.2. ПРОВЕРА ЈЕЗГРА АЛГОРИТМА

Провера језгра користи мали усмерени граф са две могуће путање. Пут преко чвора slow има трошкове 8 и 8, укупно 16. Пут преко fast има трошкове 2 и 3, укупно 5. Тест захтева да алгоритам врати редослед start, fast, goal и трошак 5. Други захтев тражи повратак од goal до start, за који у графу нема усмерених грана, па очекивани резултат није пронађен пут.

Постојећи тест је успешно извршен 22. септембра 2026. године након превођења TypeScript сервера. Он потврђује основни избор јефтиније путање, реконструкцију и понашање без доступне везе. Не испитује сва OSM правила, сваку врсту мрежне грешке ни све географске области. Његова вредност је у контролисаном очекивању: исправан исход може се израчунати унапред и непосредно упоредити са резултатом функције.

9.3. ПРОВЕРА НАД СТВАРНИМ ГЕОГРАФСКИМ ПОДАЦИМА

Интеграциона провера истог дана позвала је стварни routingService између координата 44,8116° N, 20,4618° E и 44,8150° N, 20,4650° E у Београду. Сервер је преузео податке из OpenStreetMap-а и вратио ознаку algorithm: a-star. Изабрана линија имала је 81 тачку, дужину 995 m и основну процену два минута, са опсегом 2–4 минута. Табела 9.1 издваја резултате.

Табела 9.1. Резултат једне интеграционе провере A* руте.

Величина	Добијена вредност
Ознака алгоритма	a-star
Број тачака линије	81

Величина	Добијена вредност
Дужина	995 m
Процењено време	2 min
Опсег	2–4 min
Први позив	2342 ms
Поновљени позив са кешираним графом	8 ms

Поновљени позив у истом процесу вратио је исти резултат и трајао је осам милсекунди. Први позив трајао је 2.342 ms и обухватио је и учитавање података. Ова разлика илуструје утицај кеша, али једно мерење није довољно за закључак о просечним перформансама система. Време зависи од мреже, густине графа, процесора и текућег оптерећења. Ни добијена процена од два минута није потврда стварног времена вожње, јер у овој провери није извршена физичка испорука.

9.4. ОГРАНИЧЕЊА И ПРАВЦИ УНАПРЕЂЕЊА

Најважнија ограничења потичу из модела пута. Филтрирање OSM ознака је делимично, забране скретања нису обрађене, а приступне линије нису проверене као уличне гране. Величина исечка ограничава расположиве алтернативе. За бољу навигацију потребни су прецизније прикључење на сегмент, разматрање више приступних чворова и богатији модел саобраћајних правила. Претрага је исправна у односу на дати граф, док практична употребљивост зависи и од квалитета тог графа.

Други правац је усклађивање времена. Статусни модел и модел вожње тренутно имају различите улазе и значења. Јединствена процена могла би да користи преосталу припрему и време доласка достављача као паралелне активности, а потом дода вожњу до купца и време предаје. Такво решење би захтевало мерења стварних догађаја и проверу грешке предвиђања. Без тих података не треба програмску резерву приказивати као емпиријски потврђену тачност.

Трећи правац је рад под већим оптерећењем. Меморијски кеш је ограничен бројем графова, али не и њиховом укупном величином. XML парсирање и A* се извршавају у серверском процесу и за велике графове могу задржати обраду других захтева. Могућа унапређења су обједињавање истовремених преузимања, просторни индекс за прикључење координата и издвајање тежег рачунања. Њихова оправданост треба да се утврди мерењем више репрезентативних рута.

10. ЗАКЉУЧАК

Серверски део Voro портала реализује повезан ток од креирања поруџбине до потврђене испоруке. Слојевита организација раздваја HTTP комуникацију, пословна правила и приступ подацима. PostgreSQL чува трајно стање и обезбеђује трансакцијско прихватање понуде, док Redis подржава краткотрајну присутност, просторну претрагу и ред послова. Позадински процес доделе омогућава поновне покушаје без ослањања на то да корисник држи апликацију отвореном.

Примена A* остварена је локалним конструисањем и претрагом усмереног OSM графа. Тежине представљају процењена времена сегмената, а хеуристика користи географско растојање и горњу брзину модела. Приоритетни ред, мапа трошкова и претходника омогућавају избор и реконструкцију пута. Употреба истог сервиса за понуду, активну испоруку и симулацију даје заједничку основу за координате, дужину и време вожње.

Избор достављача остаје засебан поступак заснован на доступности, удаљености и оптерећењу. Паралелне понуде убрзавају тражење извршиоца, а трансакција спречава двоструку доделу. Праћење локације допуњује тај ток провером свежине, квалитета и близине одредишта. Локална симулација користи исте сервисе и омогућава демонстрацију без физичког кретања, уз очуване услове за промену статуса.

Извршене провере потврдиле су избор јефтинијег пута у контролисаном графу и добијање стварне A* руте над подацима Београда. Резултати показују функционалну интеграцију, док ограничења исечка, приступних сегмената и брзинског модела одређују домет оптималности и процене времена. Следећи развој може се усмерити на тачнији модел путних правила, мерење стварних трајања и усклађивање оперативне процене са преосталим фазама поруџбине.

ЛИТЕРАТУРА

- [1] Express, Routing, званична документација. <https://expressjs.com/en/guide/routing/> (приступљено 22. 9. 2026).
- [2] R. T. Fielding, Architectural Styles and the Design of Network-based Software Architectures, докторска дисертација, University of California, Irvine, 2000, поглавље 5. https://ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm (приступљено 22. 9. 2026).
- [3] PostgreSQL Global Development Group, PostgreSQL 16 Documentation, Transactions. <https://www.postgresql.org/docs/16/tutorial-transactions.html> (приступљено 22. 9. 2026).
- [4] Redis, Redis geospatial, званична документација. <https://redis.io/docs/latest/develop/data-types/geospatial/> (приступљено 22. 9. 2026).
- [5] OpenStreetMap Wiki, API v0.6, Retrieving map data by bounding box, документација заједнице. https://wiki.openstreetmap.org/wiki/API_v0.6 (приступљено 22. 9. 2026).
- [6] P. E. Hart, N. J. Nilsson, B. Raphael, “A Formal Basis for the Heuristic Determination of Minimum Cost Paths”, IEEE Transactions on Systems Science and Cybernetics, SSC-4, бр. 2, стр. 100–107, 1968. DOI: 10.1109/TSSC.1968.300136. <https://ai.stanford.edu/~nilsson/OnlinePubs-Nils/PublishedPapers/astar.pdf>
- [7] Socket.IO, Rooms, званична документација, верзија 4. <https://socket.io/docs/v4/rooms/> (приступљено 22. 9. 2026).